

Ejercicios REST — Nivel Inicial

API base: NestJS con módulo `products` (CRUD en memoria) y módulo `users` (proxy a JSONPlaceholder). Archivos de referencia: `src/products/`, `src/users/`

Ejercicio 1 — Filtrar productos por nombre

Objetivo: Agregar búsqueda por query param al endpoint existente `GET /products`.

Lo que tiene que hacer: - `GET /products?name=cafe` debe devolver solo los productos cuyo nombre contenga “cafe” (case-insensitive) - Si no se pasa `?name`, sigue devolviendo todos

Archivos a modificar: - `src/products/controllers/products.controller.ts`
→ leer `@Query('name')` - `src/products/services/products.service.ts` →
pasar el filtro al repositorio - `src/products/repositories/in-memory-products.repository.ts`
→ filtrar el array

Resultado esperado:

`GET /products?name=leche` → `[{ id: 2, name: "Leche", price: 150 }]`
`GET /products` → todos los productos

Ejercicio 2 — Agregar campo stock al producto

Objetivo: Extender el modelo `Product` con un campo numérico `stock`.

Lo que tiene que hacer: - El tipo `Product` en `product.types.ts` debe incluir `stock: number` - `CreateProductInput` debe pedir `stock` - `UpdateProductInput` debe permitir actualizar `stock` (opcional) - El repositorio debe persistir y devolver ese campo

Archivos a modificar: - `src/products/product.types.ts` - `src/products/repositories/in-memory-pro`

Resultado esperado:

`POST /products` body: `{ name: "Yerba", price: 500, stock: 100 }`
`GET /products/1` → `{ id: 1, name: "Yerba", price: 500, stock: 100 }`

Ejercicio 3 — Endpoint para reducir stock

Objetivo: Crear un nuevo endpoint que descuenta unidades del stock de un producto.

Ruta: PATCH /products/:id/stock

Body: { quantity: number }

Reglas: - Si quantity es mayor al stock disponible, responder 400 Bad Request con mensaje "Stock insuficiente" - Si el producto no existe, responder 404 - Si todo está bien, actualizar el stock y devolver el producto actualizado

Archivos a modificar/crear: - src/products/controllers/products.controller.ts
→ nuevo método con @Patch('/:id/stock') - src/products/services/products.service.ts
→ lógica de validación - src/products/repositories/in-memory-products.repository.ts
→ método updateStock

Ejercicio 4 — Obtener un usuario por ID

Objetivo: Extender el módulo users para obtener un usuario individual desde JSONPlaceholder.

Ruta nueva: GET /users/:id

La API externa disponible: <https://jsonplaceholder.typicode.com/users/:id>

Reglas: - Si JSONPlaceholder devuelve 404, tu API también debe devolver 404 Not Found - Mantener el mismo patrón de manejo de errores que UsersService ya usa

Archivos a modificar: - src/users/gateways/users.gateway.ts → agregar método fetchById(id: number) - src/users/gateways/jsonplaceholder-users.gateway.ts → implementar - src/users/services/users.service.ts → nuevo método findOne - src/users/controllers/users.controller.ts → nuevo @Get('/:id')

Ejercicio 5 — Listar productos ordenados

Objetivo: Agregar parámetro de ordenamiento al listado de productos.

Query params a soportar: - ?orderBy=price → ordenar por precio - ?orderBy=name → ordenar por nombre - ?order=asc o ?order=desc (default: asc)

Resultado esperado:

GET /products?orderBy=price&order=desc
→ productos de mayor a menor precio

Archivos a modificar: - src/products/controllers/products.controller.ts
- src/products/services/products.service.ts - src/products/repositories/in-memory-products.rep